



Ripple

Git shows what changed. Ripple shows which data products changed meaning, by how much, and whether that contradicts the original intent.

Entire Graph

Entire Checkpoints

Databricks: Unity Catalog + SQL warehouse

Python stdlib only

The reviewer of a data pipeline change who has to say **BLOCK** or **PASS**.

A data engineer, or the human behind a coding agent, looking at a small diff in a transformation pipeline minutes before it ships.

What that person needs

- **Which tables and dashboards** this diff reaches, not just which functions.
- **How much the output moves** on the same input, row by row and in aggregate.
- **What the code was supposed to guarantee**, next to the evidence that it no longer does.
- **Something they can re-query** themselves rather than trust a tool's summary.

SECONDARY STAKEHOLDER Finance, the consumer of the "Daily Revenue dashboard" and the "month-end close extract". They never see the diff, only its consequences.

A harmless-looking diff. Tests pass. Revenue is overstated by 30%.

```
pipeline/transactions.py  normalize_amount()

-     magnitude = round(float(amount), 2)
+     # Normalize transaction amounts: coerce to a clean non-negative magnitude.
+     magnitude = round(abs(float(amount)), 2)
+     if kind == "refund":
-         return -magnitude
+         return abs(-magnitude)
+     return magnitude
```

The documented intent, recorded in an Entire checkpoint, says **refunds must remain negative** because daily net revenue is `sum(amount)`. The agent "normalizing amounts" flipped every refund positive.

13,854.10

net revenue, base

18,033.82

net revenue, head

33 / 150

rows changed, exactly the refunds

13 / 14

days with a different total

One command. Seven steps. One review card.

1 · GRAPH

entire graph diff

base..head → changed symbols, e.g.
`normalize_amount`

2 · GRAPH

entire graph impact

callers and data flows → pipeline entry points

3 · CONFIGURED

ripple.toml mapping

entry point → Databricks job, input and output tables, data products

4 · CHECKPOINT

Intent lookup

INTENT.md plus `entire checkpoint` for the affected pipeline

5 · EXECUTE

Run base and head

both versions over the same input snapshot

6 · DATABRICKS

Compare in SQL

row-level and aggregate diff on the warehouse, tagged with a `run_id`

7 · CARD

Review card

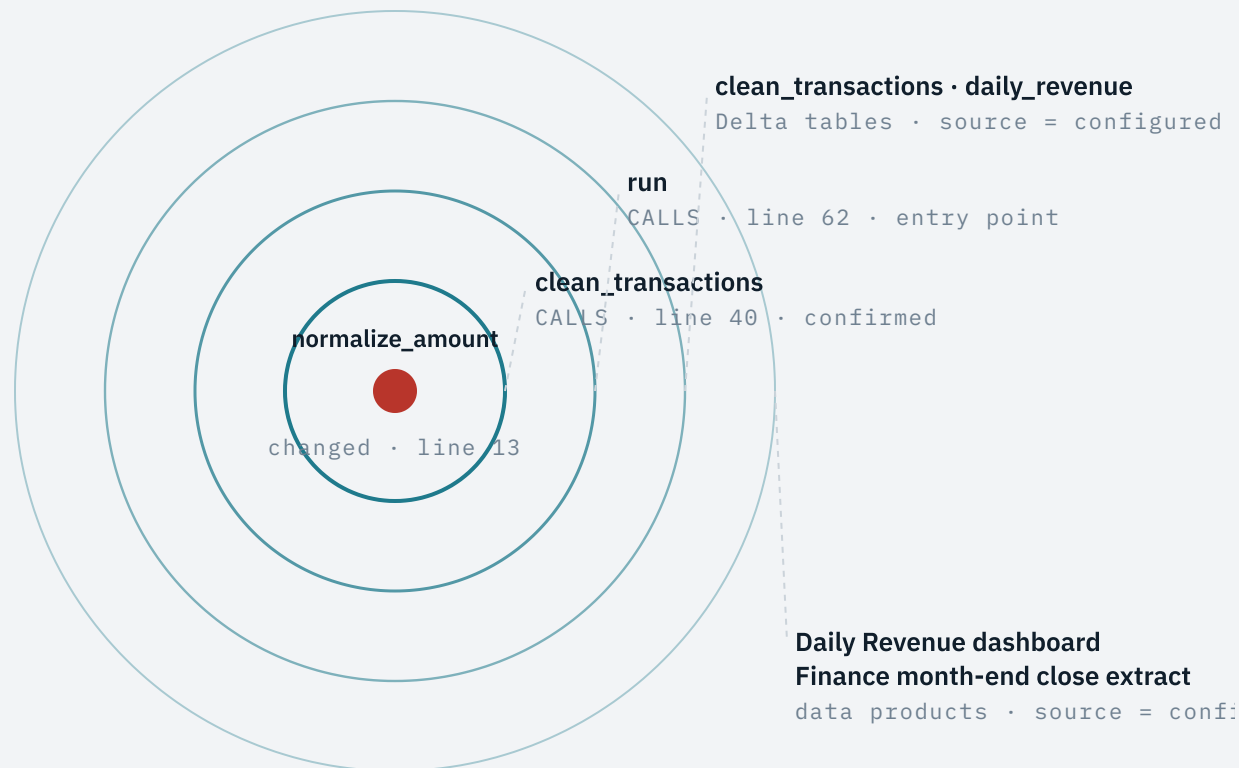
intent, source lines, data products, reproducible comparison, verdict

Scope chosen for the event window

- One Python pipeline, three Delta tables, one aggregate: daily net revenue.
- One seeded regression: `abs()` introduced while "normalizing amounts".
- Source-to-job mapping is configured, not discovered, and labelled as such on the card.

Why Entire is essential

- Without the graph, Ripple would re-run every pipeline on every change.
- Without checkpoints, the card could show a delta but not the requirement it violates.
- Graph output is treated as evidence, not an oracle. Every relation carries a confidence label.



Graph edges reach the entry point. Configuration reaches the data.

The two inner rings come from `entire_graph_impact` and are labelled `[CALLS, entire-graph]` with their call-site line. The two outer rings come from `ripple.toml` and are labelled `source = configured`. The card never blurs the

evidence/review_card_demo.md · main → demo/abs-normalize

BLOCK downstream data changed meaning

1 · ORIGINAL REQUIREMENT (INTENT)

Refunds must remain negative. Daily net revenue is `sum(amount)` and is only correct if this holds. Source: `pipeline/INTENT.md`, checkpoint `01M1TVFK...`

2 · SOURCE LOCATIONS (ENTIRE-GRAPH)

- `normalize_amount` · transactions.py:13 · body_changed
- depth 1: `clean_transactions` · call site line 40 · **confirmed**
- depth 2: `run` via `clean_transactions` · call site line 62 · **confirmed**

3 · AFFECTED DATA PRODUCTS (CONFIGURED)

workspace.ripple.clean_transactions, workspace.ripple.daily_revenue → Daily Revenue dashboard, Finance month-end close extract

4 · REPRODUCIBLE COMPARISON

- T0001 amount `-62.79` → 62.79 (and 32 more refund rows)
- 2026-08-12 net_revenue `-135.29` → 135.29
- Databricks run_id `main..demo/abs-normalize@20260906T083720Z`, three SQL statement ids

Anyone can run it,
read it, and re-query it.

Zero callers and "completeness: ok" can both be true while the graph is wrong.

We rerouted the same pipeline through a string registry and `globals()`. The real graph output: **0 callers, completeness ok, no partial failures**. The MVP would have returned PASS without executing anything. The graph does not know what it missed, so its own signals cannot flag it.

```
tests/fixtures/transactions_dynamic_dispatch.py
STEP_REGISTRY = {"normalize": "normalize_amount", ...}
fn = globals()[STEP_REGISTRY[step]]
```

PARTIAL ANALYSIS. The call graph may be missing callers of the changed code. Graph relationships are labelled per edge; nothing here is PASS on graph evidence alone.

- dynamic reference at transactions.py:13 (symbol name as string literal)
- dynamic reference at transactions.py:26 (reflection via `globals()`)
- Fallback: executed `pipeline.transactions:run` even though the graph did not reach it.

Evidence: analysis partial; 2 confirmed, 0 heuristic, 2 needs-verification

The BLOCK decision is a SQL result a reviewer can re-run.

What runs where

- Snapshot and both outputs land in Unity Catalog Delta tables under `workspace.ripple`, tagged with a `run_id`.
- Row-level diff, aggregate diff and totals are computed on the serverless SQL warehouse through the Statement Execution API.
- The three statement ids and the SQL are printed on the card and saved to `evidence/databricks/last_run.json`.

```
-- daily_revenue, base vs head, same run
SELECT b.txn_date,
       b.net_revenue AS base,
       h.net_revenue AS head,
       h.net_revenue - b.net_revenue AS diff
FROM   workspace.ripple.daily_revenue b
JOIN   workspace.ripple.daily_revenue h
      ON b.txn_date = h.txn_date
WHERE  b.run_id = '<run_id>' AND b.version = h.version
      AND h.run_id = '<run_id>' AND h.version = b.version
      AND b.net_revenue <> h.net_revenue;

-- 2026-08-12:  -135.29 → 135.29  (delta)
```

Honest scope

- The two Python versions execute locally from git revisions. Databricks stores and compares. Running the pipeline as a Databricks job is the next step.
- Free Edition, one 2X-Small warehouse, about 1 minute cold start. Whole review about 70 s.
- Inserts use literal VALUES: fine for 150 rows, not for millions.
- If the CLI or warehouse fails, the card falls back to the local comparison and says so.

What we built, what we used, and who wrote it.

CATEGORY	DETAIL
Prior work	None. Empty git history at 13:10 on event day. No prior codebase, no template.
External services	Entire CLI with the Graph plugin v0.4.0 (diff, impact, search, checkpoints). Databricks Free Edition: Unity Catalog, serverless SQL warehouse, Statement Execution API via the <code>databricks</code> CLI. GitHub for the repo mirror.
Open-source dependencies	Python 3.11 standard library only. No pip packages. <code>git</code> .
AI-generated components	The product code, tests and docs were written by coding-agent sessions (Claude Code) directed by the team, in four checkpointed sessions. Every session is recorded as an Entire checkpoint and in <code>progress.md</code> . The seeded regression on <code>demo/abs-normalize</code> is deliberately hand-specified.
Data	<code>data/raw_transactions.csv</code> is synthetic: seed 2026, 150 rows, generated by <code>data/generate_snapshot.py</code> . No real or personal data.
Hand-configured	<code>ripple.toml</code> maps entry point → job, tables, data products and intent. Labelled <code>source = configured</code> on every card.

- CHECKPOINT 1Intent and architecture before any code. Commit `ab574b4`.
- CHECKPOINT 2MVP end to end, 15 tests. Commit `97e3307`, Entire `01M1TY0G8KHGFN783A6VVB00QR`.
- CHECKPOINT 3Curveball response, 37 tests. Commit `8cf3d61`, Entire `01M1TYWW6PB644YRN2KZXMKCP3`.
- CHECKPOINT 4Final verification and semantic diff of the response. 57 entity changes. Commit `3f24ec9`, Entire `01M1TYYB3Y20A3NY7Y5FXF0FM4`.

Known limits today, and the next concrete step for each.

LIMITATION NOW	NEXT STEP
Dynamic-reference scan is Python-only and same-file: a name in YAML dispatched from another module is not caught (the graph-signal path still applies).	Scan configured entry files for string constants naming any changed symbol across files; add a CONFIGURES-edge check.
<code>entire graph</code> indexes the working tree, so impact for a head revision that is not checked out reflects the checked-out code. The demo uses a worktree.	Run the impact query inside a temporary worktree at <code>head</code> automatically.
Both pipeline versions execute locally. Databricks stores and compares.	Run the pipeline as a Databricks job so the fallback executes there too, and batch inserts for large snapshots.
Entry point → data product mappings are hand-configured in <code>ripple.toml</code> .	Read Unity Catalog lineage to discover them, and label those edges with their own evidence level.
One pipeline, one aggregate, one warehouse.	Multi-pipeline repos, a CI gate on the exit code, and the card posted as a PR comment.

What it takes to become a reliable product: automatic worktree indexing, job-based execution on Databricks, lineage-driven mappings, and a CI integration. Each is a bounded change to one module of a 1,000-line package.

Requirement, code location, and data evidence on one card. Every relationship labelled by how much to trust it.

01

User

The reviewer who must say BLOCK or PASS on a pipeline change.

02

Problem

Diff's look harmless; downstream data silently changes meaning.

03

Solution

Graph diff → impact → intent → execute both → compare on Databricks.

04

Working core

One command, 37 tests, three live cards, re-queryable SQL.

05

Transparency

Stdlib only, synthetic data, agent-written code, four checkpoints.

06

Continuation

Worktree indexing, Databricks jobs, lineage, CI gate.

```
git clone <repo> && cd buildathon-2026
python3 -m unittest discover -s tests          # 37 OK
python3 -m ripple review --base main --head demo/abs-normalize --backend databricks
```